

4장. 시맨틱 HTML과 웹 접근성

학습 목표 - 시맨틱 태그로 의미가 명확한 페이지 구조를 설계할 수 있습니다 - 문서 아웃라인과 랜드마크의 개념을 설명할 수 있습니다 - 접근성을 고려하여 키보드와 스크린 리더 친화적인 페이지를 만들 수 있습니다 - 기본적인 ARIA 속성과 SEO 메타 정보를 적용할 수 있습니다

4-1. 시맨틱 태그로 의미^만 부여하기 디자인하지는 x

도입

HTML에서 `<div>` 만으로 레이아웃을 짜는 방식을 흔히 "divitis"라고 부릅니다. 이 방식은 화면에는 아무 문제 없이 보이지만, 그 코드를 읽는 사람과 기계 모두에게 의미를 전달하지 못합니다. 시맨틱(Semantic) 태그를 사용하면 태그 이름 자체가 콘텐츠의 역할을 설명하여, 검색엔진과 보조기기, 그리고 미래의 동료 개발자가 문서를 훨씬 쉽게 이해할 수 있게 됩니다.

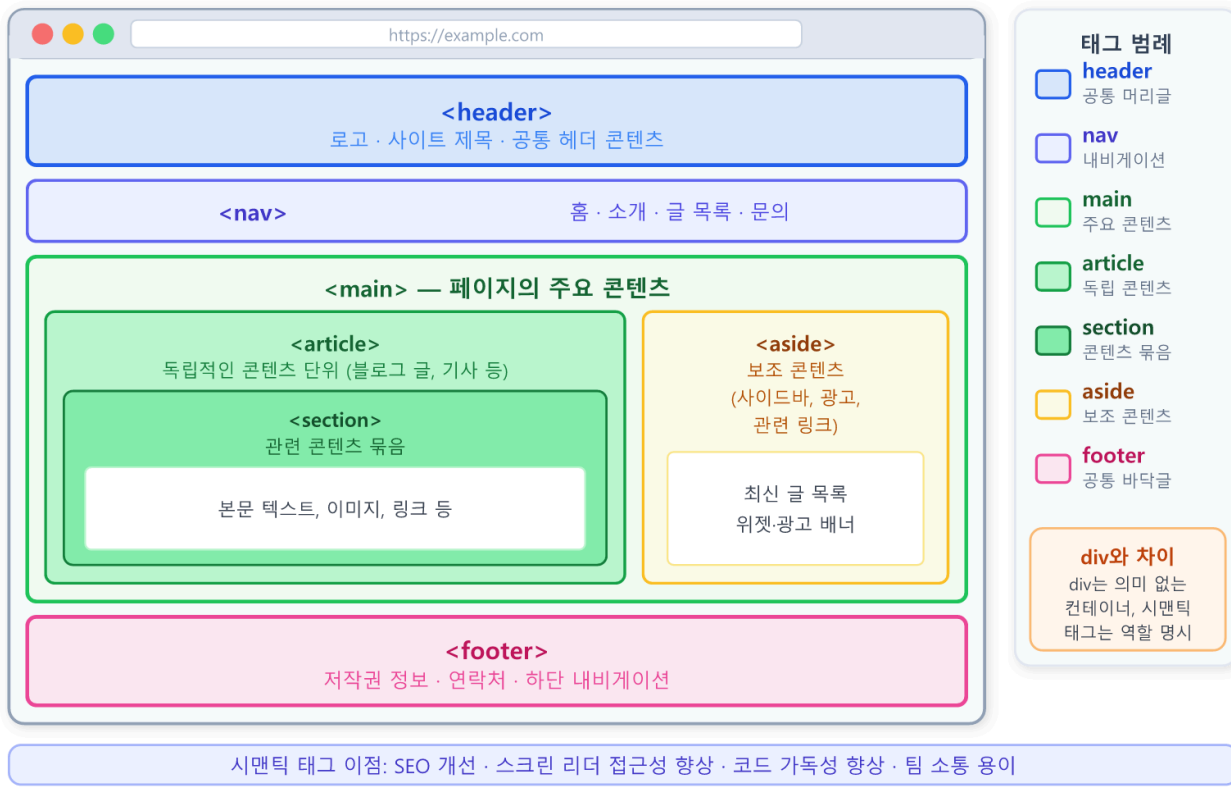
핵심 개념 설명

시맨틱 태그 (Semantic Tags)

서랍마다 "양말", "셔츠"라는 라벨을 붙여 두면 누구나 내용을 바로 알 수 있습니다. **시맨틱 태그(Semantic Tags)**는 HTML 요소에 붙은 바로 그 라벨입니다. `<header>`, `<nav>`, `<main>`, `<article>`, `<footer>` 처럼 콘텐츠의 의미와 역할을 이름으로 드러내는 태그를 말합니다.

반면 `<div>` 와 `<span>` 은 의미가 없는 컨테이너입니다. 레이아웃이나 스타일링을 위해 요소를 묶을 때는 여전히 유용하지만, 콘텐츠의 역할을 나타낼 때는 적절한 시맨틱 태그를 우선해야 합니다.

시맨틱 태그(Semantic Tag)로 구획한 웹 페이지 레이아웃



시맨틱 태그로 구획한 웹 페이지 레이아웃 와이어프레임

div와 시맨틱 태그의 차이

시각적으로는 동일하게 보이지만, 의미 전달 능력은 전혀 다릅니다.

div만 사용한 구조 (권장하지 않음):

```
<div id="header">
  <div id="nav"> ... </div>
</div>
<div id="main">
  <div id="article"> ... </div>
  <div id="sidebar"> ... </div>
</div>
<div id="footer"> ... </div>
```

이 코드는 `id` 이름에 의미를 담으려 하지만, 기계(스크린 리더, 검색엔진)는 `div` 를 단순한 상자만으로 인식합니다.

시맨틱 태그를 사용한 구조 (권장):

```

<header>
  <nav> ... </nav>
</header>
<main>
  <article> ... </article>
  <aside> ... </aside>
</main>
<footer> ... </footer>

```

태그 이름 자체가 "이것은 헤더, 이것은 내비게이션, 이것은 기사 본문"임을 명시합니다.

header, nav, main, footer로 큰 영역 나누기

페이지의 큰 틀을 잡는 네 가지 태그를 먼저 익힙니다.

태그	역할	사용 위치
<code><header></code>	소개·로고·제목·주요 내비게이션 포함	페이지 상단 또는 <code><article></code> 상단
<code><nav></code>	사이트의 주요 내비게이션 링크 모음	<code><header></code> 안 또는 독립적으로
<code><main></code>	페이지의 핵심 콘텐츠	문서당 단 하나 만 사용
<code><footer></code>	저작권·연락처·관련 링크 등 마무리 정보	페이지 하단 또는 <code><article></code> 하단

주의: `<main>` 요소는 페이지당 하나만 사용합니다. 브라우저와 스크린 리더는 `<main>` 을 페이지의 핵심 영역으로 인식하므로, 두 개 이상 존재하면 혼란을 줍니다.

section, article, aside의 적절한 사용

더 세밀한 콘텐츠 영역을 나눌 때 사용하는 세 가지 태그입니다.

- `<article>` : 독립적으로 배포하거나 재사용할 수 있는 완결된 콘텐츠. 블로그 글, 뉴스 기사, 댓글, 제품 카드 등이 해당됩니다. "이 내용만 떼어내도 의미가 통하는가?"라고 스스로 질문해 보십시오.
- `<section>` : 주제별로 콘텐츠를 구분할 때 사용합니다. 독립적이지는 않지만 제목이 있는 구분된 영역이라면 `<section>` 이 적합합니다.

- `<aside>`: 주 콘텐츠와 간접적으로 관련된 부가 정보. 사이드바, 관련 글 링크, 광고 등이 해당됩니다.

코드로 확인하기

다음 코드는 시맨틱 태그를 활용하여 블로그 글 페이지의 뼈대를 구성하는 예입니다. `<div>` 없이도 구조가 명확하게 표현됩니다.

```
<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>시맨틱 HTML 블로그 예제</title>
</head>
<body>

  <!-- 페이지 상단 헤더 영역 -->
  <header>
    <h1>나의 기술 블로그</h1>
    <nav>
      <ul>
        <li><a href="#">홈</a></li>
        <li><a href="#">글 목록</a></li>
        <li><a href="#">소개</a></li>
      </ul>
    </nav>
  </header>

  <!-- 핵심 콘텐츠 (페이지당 하나) -->
  <main>

    <!-- 독립적으로 완결된 블로그 글 -->
    <article>
      <header>
        <h2>시맨틱 HTML을 써야 하는 이유</h2>
        <p>작성일: <time datetime="2025-06-01">2025년 6월 1일</time></p>
      </header>

      <section>
        <h3>검색엔진에게 의미를 전달한다</h3>
        <p>구글은 시맨틱 태그를 통해 페이지 구조를 파악합니다.</p>
      </section>

      <section>
        <h3>스크린 리더가 탐색하기 쉬워진다</h3>
        <p>시각 장애인 사용자는 랜드마크를 기준으로 페이지를 탐색합니다.</p>
      </section>
    </article>

    <!-- 주 콘텐츠와 관련된 부가 정보 -->
    <aside>
      <h2>관련 글</h2>
      <ul>
        <li><a href="#">웹 접근성 가이드라인</a></li>
        <li><a href="#">SEO 기초 정리</a></li>
      </ul>
    </aside>
```

```

</main>

<!-- 페이지 하단 영역 -->
<footer>
  <p>&copy; 2025 나의 기술 블로그. All rights reserved.</p>
</footer>

</body>
</html>

```

브라우저에서 열면 시각적으로는 제목과 목록이 나열됩니다. CSS를 아직 적용하지 않았기 때문에 스타일은 단순하지만, 구조 자체는 검색엔진과 스크린 리더가 완벽하게 이해할 수 있는 형태입니다.

줄별 코드 설명

줄	코드	설명
3	<code><html lang="ko"></code>	문서 언어를 한국어로 선언합니다. 스크린 리더가 올바른 음성 엔진을 선택하고 검색엔진이 언어를 파악하는 데 사용됩니다
12	<code><header></code>	페이지 전체의 상단 영역을 나타냅니다. 로고, 사이트명, 주요 내비게이션을 담습니다
14	<code><nav></code>	사이트 내 주요 이동 링크 모음을 의미합니다. 스크린 리더 사용자가 이 영역을 건너뛰거나 바로 이동할 수 있습니다
22	<code><main></code>	이 페이지의 핵심 콘텐츠 영역입니다. 문서당 하나만 존재해야 합니다
25	<code><article></code>	독립적으로 완결된 블로그 글 본체입니다. 이 태그 안의 <code><header></code> 는 글 자체의 헤더(제목, 날짜)를 나타냅니다
28	<code><time datetime="2025-06-01"></code>	기계가 읽을 수 있는 날짜 형식(ISO 8601)을 <code>datetime</code> 속성에 지정합니다
32	<code><section></code>	글 안에서 주제별로 구분된 하위 섹션입니다. 제목 (<code><h3></code>)과 함께 사용합니다
43	<code><aside></code>	본문과 관련된 부가 정보(관련 글 링크)를 담습니다
52	<code><footer></code>	페이지의 마무리 정보(저작권 등)를 담습니다

팁: `<article>` 안에도 `<header>` 와 `<footer>` 를 사용할 수 있습니다. 이 경우 페이지 전체의 헤더/풋터가 아니라, 해당 `<article>` 에만 속하는 헤더/풋터를 의미합니다.

베스트 프랙티스: `<section>` 에는 항상 제목 태그(`<h2>` , `<h3>` 등)를 포함하십시오. 제목 없는 `<section>` 은 의미가 불분명해집니다.

절 요약

- 시맨틱 태그는 콘텐츠의 역할을 태그 이름으로 표현하여 검색엔진, 스크린 리더, 개발자 모두에게 의미를 전달합니다
- `<header>` , `<nav>` , `<main>` , `<footer>` 로 페이지의 큰 구획을 나눕니다
- `<article>` 은 독립 완결 콘텐츠, `<section>` 은 주제별 구분, `<aside>` 는 부가 정보에 사용합니다
- `<main>` 은 페이지당 하나만 사용합니다

4-2. 문서 아웃라인과 구조 설계

도입

잘 작성된 책에는 목차가 있고, 대제목 아래 중제목이 있으며, 중제목 아래 소제목이 이어집니다. 웹 문서도 마찬가지입니다. **문서 아웃라인(Document Outline)**은 HTML 페이지의 제목 계층 구조를 말하며, 검색엔진과 스크린 리더가 가장 먼저 파악하는 페이지의 뼈대입니다.

핵심 개념 설명

문서 아웃라인 (Document Outline)

문서 아웃라인은 페이지 안의 모든 제목 태그(`<h1>` ~ `<h6>`)를 순서대로 나열한 구조입니다. 스크린 리더 사용자는 제목 목록을 탐색 수단으로 활용하여 원하는 섹션으로 바로 이동합니다. 검색엔진은 제목 계층을 통해 페이지의 핵심 주제를 파악합니다.

올바른 제목 계층 구조: - `<h1>` : 페이지 전체의 대주제 (페이지당 하나) - `<h2>` : 주요 섹션의 제목 - `<h3>` : 서브섹션의 제목 - `<h4>` ~ `<h6>` : 더 세밀한 하위 구분

주의: 제목 레벨을 순서대로 사용하십시오. `<h1>` 다음에 `<h3>` 으로 건너뛰면 아웃라인에 구멍이 생깁니다. 글자 크기를 조절하고 싶다면 CSS를 사용하고, 제목 태그는 구조를 위해서만 사용합니다.

랜드마크 (Landmark)

랜드마크(Landmark)는 스크린 리더 사용자가 페이지 안에서 빠르게 이동할 수 있도록 돕는 구역 표지판입니다. 시맨틱 태그들이 자동으로 랜드마크 역할을 합니다.

시맨틱 태그	대응하는 ARIA 랜드마크 역할
<code><header></code>	<code>banner</code>
<code><nav></code>	<code>navigation</code>
<code><main></code>	<code>main</code>
<code><aside></code>	<code>complementary</code>
<code><footer></code>	<code>contentinfo</code>
<code><section></code> (제목 있는 경우)	<code>region</code>
<code><form></code>	<code>form</code>

스크린 리더(NVDA, VoiceOver 등)에서는 단축키 하나로 랜드마크 목록을 불러와 원하는 영역으로 즉시 이동할 수 있습니다. 시맨틱 태그를 사용하는 것만으로도 이 기능이 자동으로 지원됩니다.

코드로 확인하기

다음 코드는 올바른 제목 계층과 랜드마크 구조를 갖춘 페이지 예제입니다. 크롬 개발자 도구의 접근성 탭에서 이 구조를 확인할 수 있습니다.


```

<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8">
  <title>문서 아웃라인 예제</title>
</head>
<body>

  <header>
    <!-- h1: 페이지 전체의 대주제 (하나만 사용) -->
    <h1>웹 개발 입문 가이드</h1>
    <nav>
      <h2>목차</h2>
      <ul>
        <li><a href="#html">HTML 기초</a></li>
        <li><a href="#css">CSS 스타일링</a></li>
        <li><a href="#js">JavaScript</a></li>
      </ul>
    </nav>
  </header>

  <main>
    <article id="html">
      <!-- h2: 주요 섹션 -->
      <h2>HTML 기초</h2>
      <p>HTML은 웹 페이지의 구조를 정의합니다.</p>

      <section>
        <!-- h3: 서브섹션 -->
        <h3>시맨틱 태그</h3>
        <p>콘텐츠의 의미를 전달하는 태그를 사용합니다.</p>

        <section>
          <!-- h4: 더 세밀한 하위 구분 -->
          <h4>주요 시맨틱 태그 목록</h4>
          <ul>
            <li><code>&lt;header&gt;</code> - 헤더</li>
            <li><code>&lt;main&gt;</code> - 주요 콘텐츠</li>
            <li><code>&lt;footer&gt;</code> - 풋터</li>
          </ul>
        </section>
      </section>

      <section>
        <h3>접근성</h3>
        <p>모든 사용자가 콘텐츠에 접근할 수 있어야 합니다.</p>
      </section>
    </article>

    <article id="css">

```

```

    <h2>CSS 스타일링</h2>
    <p>CSS는 웹 페이지의 시각적 표현을 담당합니다.</p>
  </article>

</main>

<footer>
  <p>© 2025 웹 개발 입문 가이드</p>
</footer>

</body>
</html>

```

이 문서의 아웃라인은 다음과 같이 구성됩니다:

- 1 웹 개발 입문 가이드 ← h1 (페이지 대주제)
- 1.1 목차 ← h2 (nav 안의 제목)
- 1.2 HTML 기초 ← h2 (article 제목)
 - 1.2.1 시맨틱 태그 ← h3
 - 1.2.1.1 주요 시맨틱 태그 목록 ← h4
 - 1.2.2 접근성 ← h3
- 1.3 CSS 스타일링 ← h2 (article 제목)

줄별 코드 설명

줄	코드	설명
11	<code><h1>웹 개발 입문 가이드</h1></code>	페이지 전체를 대표하는 유일한 대주제입니다. 문서당 하나만 사용합니다
13	<code><h2>목차</h2></code>	<code><nav></code> 안의 섹션 제목입니다. <code><nav></code> 에도 제목을 부여하면 스크린 리더가 이 영역이 무엇인지 알 수 있습니다
23	<code><article id="html"></code>	<code>id</code> 속성을 붙여 페이지 내 앵커 링크 (<code>href="#html"</code>)의 대상으로 삼습니다
25	<code><h2>HTML 기초</h2></code>	주요 섹션의 제목입니다. <code><h1></code> 다음 레벨인 <code><h2></code> 를 사용합니다
29	<code><h3>시맨틱 태그</h3></code>	서브섹션 제목입니다. <code><h2></code> 의 하위이므로 <code><h3></code> 을 사용합니다. 레벨을 건너뛰지 않습니다

베스트 프랙티스: Chrome 브라우저의 개발자 도구(F12) → Accessibility 탭에서 "Full page accessibility tree"를 확인하면 현재 페이지의 랜드마크와 제목 계층을 한눈에 볼 수 있습니다.

절 요약

- 문서 아웃라인은 `<h1>` ~ `<h6>` 으로 구성되는 제목 계층 구조입니다
- 제목 레벨은 순서대로 사용하고 건너뛰지 않습니다
- 시맨틱 태그는 자동으로 랜드마크 역할을 가져, 스크린 리더 사용자가 페이지를 빠르게 탐색할 수 있게 합니다
- `<nav>` 에도 `<h2>` 제목을 붙이면 여러 개의 `<nav>` 가 있을 때 구분이 쉬워집니다

4-3. 웹 접근성(a11y) 기초

도입

건물에 계단만 있다면 휠체어 사용자는 들어올 수 없습니다. 경사로와 점자 안내를 함께 두면 누구나 이용할 수 있습니다. **웹 접근성(Web Accessibility, a11y)**은 장애 여부, 나이, 사용 환경에 관계없이 모든 사람이 웹을 이용할 수 있도록 보장하는 설계 원칙입니다. "a11y"는 "accessibility"의 첫 글자(a)와 끝 글자(y) 사이에 11글자가 있어 줄여 쓴 표기입니다.

핵심 개념 설명

접근성 (Accessibility)

접근성이 중요한 이유는 단순히 소수를 위한 배려가 아닙니다. 접근성을 고려한 웹은 다음 모든 사용자에게 이롭습니다.

- **시각 장애인:** 스크린 리더로 페이지를 청취합니다
- **운동 장애인:** 마우스 없이 키보드만으로 페이지를 탐색합니다
- **노안 사용자:** 글자 크기를 크게 설정하여 사용합니다
- **저사양 기기 사용자:** 이미지가 로드되지 않을 때 alt 텍스트가 필요합니다
- **일반 사용자:** 키보드 단축키, 명확한 레이블 등 접근성 기능이 편의성을 높입니다

스크린 리더(Screen Reader)가 페이지를 읽는 방식

스크린 리더(Screen Reader)는 화면의 텍스트를 음성으로 변환해 주는 소프트웨어입니다. Windows의 NVDA, macOS/iOS의 VoiceOver, Android의 TalkBack 등이 있습니다.

스크린 리더 사용자는 다음 방식으로 페이지를 탐색합니다.

1. **랜드마크 이동**: `<nav>` , `<main>` 같은 랜드마크를 단축키로 건너뛰며 이동합니다
2. **제목 탐색**: 제목 태그(`<h1>` ~ `<h6>`) 목록을 불러와 원하는 섹션으로 이동합니다
3. **링크 목록**: 페이지의 모든 링크를 목록으로 보고 이동합니다
4. **순차 탐색**: 탭(Tab) 키로 포커스 가능한 요소(링크, 버튼, 입력 필드)를 차례로 탐색합니다

키보드 내비게이션 (Keyboard Navigation)

마우스를 사용하지 못하는 사용자는 키보드만으로 모든 기능을 사용해야 합니다. 키보드 탐색의 기본 규칙은 다음과 같습니다.

키	동작
Tab	다음 포커스 가능 요소로 이동
Shift + Tab	이전 포커스 가능 요소로 이동
Enter / Space	버튼 클릭, 링크 이동, 체크박스 토글
방향키	라디오 버튼, 탭 패널, 슬라이더 조작

기본 HTML 요소(`<a>` , `<button>` , `<input>`)는 자동으로 키보드 포커스를 받습니다.

`<div>` 나 `<span>` 에 클릭 동작을 넣으면 키보드로 접근할 수 없게 되므로, 버튼 역할에는 반드시 `<button>` 태그를 사용합니다.

코드로 확인하기

다음 코드는 키보드 탐색이 가능한 내비게이션과 접근성이 갖춰진 이미지 사용 예를 보여줍니다.

```

<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8">
  <title>접근성 기초 예제</title>
  <style>
    /* 포커스 표시를 명확하게 보여줍니다 (키보드 사용자를 위함) */
    a:focus,
    button:focus {
      outline: 3px solid #0066cc;
      outline-offset: 2px;
    }
    /* 스크린 리더 전용 텍스트 (화면에는 숨김) */
    .sr-only {
      position: absolute;
      width: 1px;
      height: 1px;
      overflow: hidden;
      clip: rect(0, 0, 0, 0);
      white-space: nowrap;
    }
  </style>
</head>
<body>

  <a href="#main-content" class="sr-only">본문으로 바로 가기</a>

  <header>
    <h1>접근성 예제 페이지</h1>
    <nav aria-label="주요 메뉴">
      <ul>
        <li><a href="#">홈</a></li>
        <li><a href="#">소개</a></li>
        <li><a href="#">연락처</a></li>
      </ul>
    </nav>
  </header>

  <main id="main-content">

    <!-- 의미 있는 이미지: alt에 내용을 설명 -->
    <figure>
      
      <figcaption>그림 1. 시맨틱 태그로 구성된 웹 페이지 구조</figcaption>
    </figure>

    <!-- 장식 이미지: alt를 빈 문자열로 설정 -->

```

```


<!-- 버튼은 div 대신 button 태그 사용 -->
<button type="button">더 읽기</button>

<!-- div에 클릭을 넣으면 안 됩니다 (키보드 접근 불가) -->
<!-- 나쁜 예: <div onclick="doSomething()">클릭</div> -->

</main>

<footer>
  <p>© 2025 접근성 예제</p>
</footer>

</body>
</html>
```

줄별 코드 설명

줄	코드	설명
8-11	<code>a:focus, button:focus { outlin...</code>	키보드로 탭 이동 시 포커스 링을 명확히 표시합니다. <code>outline: none</code> 으로 제거하면 키보드 사용자가 현재 위치를 알 수 없게 됩니다
13-19	<code>.sr-only { ... }</code>	화면에는 보이지 않지만 스크린 리더는 읽는 "스크린 리더 전용" 패턴입니다. <code>display: none</code> 이나 <code>visibility: hidden</code> 은 스크린 리더도 무시하므로 이 방법을 사용합니다
23	<code><a href="#main-content" class=...</code>	스킵 내비게이션(skip navigation) 링크입니다. 탭 키를 처음 누르면 이 링크가 나타나 반복적인 메뉴를 건너뛰고 바로 본문으로 이동할 수 있게 합니다
27	<code>aria-label="주요 메뉴"</code>	페이지에 <code><nav></code> 가 여러 개일 때 각각을 구분하는 레이블입니다. 스크린 리더는 "주요 메뉴, 내비게이션 랜드마크"라고 읽습니다
38-41	<code>alt="헤더, 메인..."</code>	이미지의 내용을 구체적으로 설명합니다. "이미지", "사진" 같은 단어는 스크린 리더가 이미 "이미지"라고 읽어주므로 중복이 됩니다
44	<code>alt=""</code>	장식 목적의 이미지는 <code>alt=""</code> 로 설정합니다. 스크린 리더가 이 이미지를 완전히 건너뛸니다
47	<code><button type="button">더 읽기</...></code>	클릭 가능한 요소는 반드시 <code><button></code> 을 사용합니다. 자동으로 키보드 포커스와 Enter/Space 키 동작이 지원됩니다

주의: `outline: none` 또는 `outline: 0` 으로 포커스 스타일을 완전히 제거하는 것은 접근성 위반입니다. 키보드 사용자가 현재 포커스 위치를 알 수 없게 됩니다. 기본 아웃라인의 외관이 마음에 들지 않는다면 커스텀 스타일로 대체하되 반드시 시각적으로 구별 가능하게 만드십시오.

베스트 프랙티스: 모든 이미지에는 `alt` 속성이 있어야 합니다. 의미 있는 이미지는 내용을 설명하고, 장식 이미지는 `alt=""` 로 처리합니다. `alt` 속성 자체가 없으면 스크린 리더가 파일명을 그대로 읽습니다.

alt와 label의 접근성 역할 복습

3장에서 배운 `alt` 와 `label` 을 접근성 관점에서 정리합니다.

```
<!-- alt: 이미지 대체 텍스트 -->


<!-- label: 입력 필드의 레이블 (for와 id 연결) -->
<label for="user-email">이메일 주소</label>
<input type="email" id="user-email" name="email">

<!-- label로 감싸는 방법 (for/id 없이도 연결됨) -->
<label>
  <input type="checkbox" name="agree">
  이용약관에 동의합니다
</label>
```

`<label>` 이 없으면 스크린 리더는 입력 필드에 포커스가 갔을 때 무슨 정보를 입력해야 하는지 알리지 못합니다. `placeholder` 는 시각적으로 보이지만 스크린 리더 지원이 불완전하고 입력 시 사라지므로 `label` 을 대체할 수 없습니다.

절 요약

- 웹 접근성은 장애인뿐 아니라 모든 사용자의 경험을 향상시킵니다
- 스크린 리더 사용자는 랜드마크, 제목, 링크 탐색을 주로 사용합니다
- 키보드 포커스 스타일을 제거하면 안 됩니다
- 스킵 내비게이션 링크는 반복 메뉴를 건너뛰는 키보드 접근성의 기본입니다
- 의미 있는 이미지에는 내용을 설명하는 `alt` , 장식 이미지에는 `alt=""` 를 사용합니다

4-4. ARIA 속성 다루기

도입

시맨틱 HTML만으로는 표현하기 어려운 상황이 있습니다. 예를 들어, JavaScript로 동적으로 열리고 닫히는 모달 창이나 탭 패널처럼 복잡한 UI 컴포넌트는 HTML 태그만으로는 현재 상태를 보조기기에 전달하기 어렵습니다. 이때 **ARIA(Accessible Rich Internet Applications)**를

사용합니다. 표지판이 부족한 곳에 추가 안내문을 붙여 길을 명확히 알려 주는 것처럼, ARIA는 HTML에 부가 의미 정보를 덧붙입니다.

핵심 개념 설명

ARIA의 황금 원칙

ARIA를 배우기 전에 가장 중요한 원칙을 먼저 이해해야 합니다.

ARIA 제1원칙: 적절한 HTML 시맨틱이 있다면 ARIA를 사용하지 마십시오.

`<button>` 으로 표현할 수 있는 것을 `<div role="button">` 으로 만들지 않습니다. 시맨틱 HTML이 모든 것을 처리할 수 있다면 ARIA는 필요하지 않습니다. ARIA는 시맨틱 HTML로 해결하기 어려운 경우에만 보완적으로 사용합니다.

role 속성으로 의미 보완하기

`role` 속성은 요소의 역할을 보조기기에 전달합니다. 불가피하게 `<div>` 로 상호작용 요소를 만들어야 할 때 사용합니다.

```
<!-- 피하는 것이 좋은 방법 (하지만 불가피한 경우) -->
<div role="button" tabindex="0">닫기</div>

<!-- 항상 이 방법을 우선합니다 -->
<button type="button">닫기</button>
```

`<div role="button">` 을 선택했다면 `tabindex="0"` 을 반드시 추가하여 키보드 포커스를 받을 수 있게 해야 합니다. 그리고 JavaScript로 Enter와 Space 키 이벤트도 직접 처리해야 합니다. 결국 `<button>` 하나로 해결되는 것을 `<div>` 로 구현하면 훨씬 더 많은 작업이 필요합니다.

aria-label과 aria-hidden 활용

자주 사용하는 두 가지 ARIA 속성입니다.

aria-label : 요소의 이름을 직접 지정합니다. 화면에 보이는 텍스트 레이블이 없거나, 있더라도 보조기기에 다른 설명이 필요할 때 사용합니다.

aria-hidden : **"true"** 로 설정하면 스크린 리더가 해당 요소를 완전히 무시합니다. 장식 아이콘, 중복 텍스트를 숨길 때 유용합니다.

코드로 확인하기

다음 코드는 ARIA 속성의 주요 사용 사례를 보여줍니다. 아이콘 버튼, 여러 내비게이션 구분, 상태 표시 영역을 다룹니다.

```

<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8">
  <title>ARIA 속성 예제</title>
</head>
<body>

  <header>
    <!-- 여러 nav가 있을 때 aria-label로 구분합니다 -->
    <nav aria-label="주요 메뉴">
      <ul>
        <li><a href="#">홈</a></li>
        <li><a href="#">서비스</a></li>
      </ul>
    </nav>
  </header>

  <main>

    <!-- 아이콘만 있는 버튼: aria-label로 목적을 설명합니다 -->
    <button type="button" aria-label="장바구니 열기">
      <!-- 아이콘 이미지나 SVG -->
      
    </button>

    <!-- 닫기 버튼: X 기호는 목적을 설명하지 못합니다 -->
    <button type="button" aria-label="모달 닫기">
      <span aria-hidden="true">&times;</span>
    </button>

    <!-- aria-expanded: 열림/닫힘 상태를 전달합니다 -->
    <button
      type="button"
      aria-expanded="false"
      aria-controls="dropdown-menu"
    >
      메뉴 열기
    </button>
    <ul id="dropdown-menu" hidden>
      <li><a href="#">옵션 1</a></li>
      <li><a href="#">옵션 2</a></li>
    </ul>

    <!-- aria-live: 동적으로 업데이트되는 영역을 알립니다 -->
    <div aria-live="polite" aria-atomic="true" id="status-message">
      <!-- JavaScript로 메시지가 업데이트되면 스크린 리더가 읽습니다 -->
    </div>

    <!-- 진행 표시줄 -->

```

```
<div
  role="progressbar"
  aria-valuenow="70"
  aria-valuemin="0"
  aria-valuemax="100"
  aria-label="파일 업로드 진행률"
>
  <div style="width: 70%; background: #0066cc; height: 20px;"></div>
</div>

</main>

<footer>
  <!-- 연도 자동 갱신 아이콘은 장식이므로 숨깁니다 -->
  <p>
    <span aria-hidden="true">©</span>
    Copyright 2025
  </p>
</footer>

</body>
</html>
```

출별 코드 설명

줄	코드	설명
10	<code>aria-label="주요 메뉴"</code>	이 <code><nav></code> 의 이름을 "주요 메뉴"로 지정합니다. 페이지에 <code><nav></code> 가 여러 개라면 각각 <code>aria-label</code> 로 구분해야 스크린 리더가 어느 내비게이션인지 알려줍니다
20-22	<code>aria-label="장바구니 열기"</code>	버튼에 텍스트가 없고 아이콘만 있을 때 사용합니다. 내부 <code></code> 의 <code>alt=""</code> 는 스크린 리더가 이미지를 건너뛰게 하여 버튼 레이블 (<code>aria-label</code>)만 읽힙니다
25-27	<code>&time...</code>	<code>&times;</code> (x) 기호는 장식적 문자입니다. <code>aria-hidden="true"</code> 로 숨기고, 버튼 자체의 <code>aria-label="모달 닫기"</code> 로 목적을 전달합니다
30-34	<code>aria-expanded="false"</code>	토글 버튼의 현재 상태를 알립니다. JavaScript로 메뉴가 열리면 <code>"true"</code> 로 변경합니다. <code>aria-controls</code> 는 이 버튼이 어떤 요소를 제어하는지 <code>id</code> 로 연결합니다
40-41	<code>aria-live="polite"</code>	이 영역의 내용이 변경되면 스크린 리더가 현재 읽기를 마친 후 변경 내용을 알립니다. <code>"assertive"</code> 는 즉시 끊고 알립니다(긴급 오류에 사용)
45-51	<code>role="progressbar"</code>	<code><div></code> 에 진행 표시줄 역할을 부여합니다. <code>aria-valuenow</code> , <code>aria-valuemin</code> , <code>aria-valuemax</code> 로 현재 값을 전달합니다

주의: `aria-hidden="true"` 를 사용할 때 그 요소 안에 포커스 가능한 요소(링크, 버튼)가 있으면 안 됩니다. 시각적으로는 숨겨지지만 키보드 탭 탐색에서는 여전히 포커스를 받는 혼란이 발생할 수 있습니다.

ARIA 남용을 피하는 원칙

ARIA 사용 시 지켜야 할 핵심 원칙 네 가지입니다.

1. **시맨틱 HTML을 우선 사용합니다.** `<button>` 대신 `<div role="button">` 을 사용하지 않습니다

2. **ARIA를 추가하면 동작도 구현합니다:** `role="button"` 을 추가했다면 키보드 이벤트도 직접 처리합니다
3. **보이는 상태와 ARIA 상태를 동기화합니다:** `aria-expanded` , `aria-checked` 등은 UI 상태와 항상 일치해야 합니다
4. **테스트합니다:** NVDA(무료, Windows), VoiceOver(macOS/iOS 내장)로 직접 탐색해 봅니다

절 요약

- ARIA는 시맨틱 HTML로 표현하기 어려운 의미와 상태를 보조기기에 전달합니다
 - `aria-label` 은 요소의 이름을, `aria-hidden="true"` 는 스크린 리더 숨김을 처리합니다
 - `aria-expanded` , `aria-live` , `role` 등으로 동적 UI의 상태를 전달할 수 있습니다
 - ARIA를 많이 사용할수록 좋은 것이 아니라, 적절한 시맨틱 HTML이 우선입니다
-

4-5. SEO 기초와 메타 정보

도입

아무리 훌륭한 웹 페이지를 만들어도 검색엔진이 찾지 못하면 방문자가 없습니다.

SEO(Search Engine Optimization, 검색엔진 최적화)는 검색 결과 상위에 페이지가 노출되도록 최적화하는 작업입니다. HTML에서 시맨틱 구조와 메타 정보는 SEO의 기초를 이룹니다.

핵심 개념 설명

검색엔진 최적화(SEO)의 기본 개념

검색엔진(구글, 네이버 등)은 웹을 순회하는 크롤러(Crawler)를 사용하여 페이지를 수집하고 분석합니다. 크롤러는 HTML 코드를 직접 읽으므로, HTML이 잘 구조화될수록 페이지 내용을 정확히 파악합니다.

SEO에 영향을 주는 HTML 요소:

- `<title>` : 검색 결과에 파란 링크 제목으로 표시됩니다
- `<meta name="description">` : 검색 결과 제목 아래 설명 문장으로 표시됩니다
- `<h1>` ~ `<h6>` : 페이지 주제와 구조를 파악하는 데 사용됩니다
- `alt` 속성: 이미지 검색에 활용됩니다

- 시맨틱 태그: 페이지 구조를 이해하는 데 도움이 됩니다
- `lang` 속성: 언어를 파악하여 적합한 검색 언어로 노출합니다

title과 meta description 작성하기

`<title>` 태그는 검색 결과의 클릭 링크 텍스트가 됩니다. `<meta name="description">` 은 제목 아래 표시되는 요약 설명입니다.

작성 가이드: - `<title>` : 50~60자 내외, 페이지 고유 키워드 포함 - `meta description` : 150~160자 내외, 클릭을 유도하는 명확한 설명 - 모든 페이지마다 고유한 title과 description 을 작성합니다

Open Graph로 공유 미리보기 설정하기

카카오톡, 트위터, 페이스북 등에 링크를 공유하면 제목, 이미지, 설명이 미리보기로 표시됩니다. 이것이 **Open Graph(OG)** 메타 태그입니다. Facebook이 만든 규격으로 현재는 사실상 표준으로 사용됩니다.

코드로 확인하기

다음 코드는 검색엔진 최적화와 소셜 공유 미리보기를 모두 갖춘 `<head>` 섹션 예제입니다.

```
<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">

  <!-- SEO: 검색 결과 제목 (50~60자 권장) -->
  <title>시맨틱 HTML 완전 정복 | 웹 개발 입문 가이드</title>

  <!-- SEO: 검색 결과 설명 (150~160자 권장) -->
  <meta
    name="description"
    content="시맨틱 HTML 태그(header, nav, main, article, footer)의 의미와
    올바른 사용법을 예제와 함께 단계별로 배웁니다. 웹 접근성과 SEO를 동시에 향상시키는
  >

  <!-- SEO: 검색 로봇 지시 (기본값이 index, follow이므로 생략 가능) -->
  <meta name="robots" content="index, follow">

  <!-- Open Graph: 소셜 미디어 공유 미리보기 -->
  <meta property="og:type" content="article">
  <meta property="og:title" content="시맨틱 HTML 완전 정복">
  <meta property="og:description" content="시맨틱 HTML 태그의 의미와 올바른 사용법">
  <meta property="og:image" content="https://example.com/images/og-semantic-html">
  <meta property="og:url" content="https://example.com/blog/semantic-html">
  <meta property="og:site_name" content="웹 개발 입문 가이드">

  <!-- Twitter Card: 트위터 공유 미리보기 -->
  <meta name="twitter:card" content="summary_large_image">
  <meta name="twitter:title" content="시맨틱 HTML 완전 정복">
  <meta name="twitter:description" content="시맨틱 HTML 태그의 의미와 올바른 사용법">
  <meta name="twitter:image" content="https://example.com/images/og-semantic-html">

  <link rel="stylesheet" href="style.css">
</head>
<body>
  <!-- 페이지 본문 -->
  <header>
    <h1>시맨틱 HTML 완전 정복</h1>
  </header>
  <main>
    <article>
      <p>내용이 여기에 들어갑니다.</p>
    </article>
  </main>
</body>
</html>
```


줄별 코드 설명

줄	코드	설명
3	<code><html lang="ko"></code>	문서 언어를 선언합니다. 검색엔진이 이 페이지를 한국어 사용자에게 노출할지 판단하는데 사용됩니다
8	<code><title>시맨틱 HTML 완전 정복 ...</code>	검색 결과 페이지에서 파란 클릭 링크 텍스트로 표시됩니다. <code> </code> 으로 페이지 제목과 사이트 이름을 구분하는 관례를 따릅니다
11-15	<code><meta name="description" conte...</code>	검색 결과에서 제목 아래 회색 요약 텍스트로 표시됩니다. 클릭률에 직접 영향을 미치므로 매력적으로 작성합니다
22	<code>og:type</code>	콘텐츠 유형을 지정합니다. <code>article</code> (기사), <code>website</code> (일반 웹사이트), <code>video.movie</code> (동영상) 등을 사용합니다
25	<code>og:image</code>	공유 시 표시될 이미지 URL입니다. 최소 1200×630px 크기를 권장합니다
30	<code>twitter:card</code>	<code>summary_large_image</code> 는 큰 이미지 미리보기, <code>summary</code> 는 작은 정사각형 이미지를 표시합니다

팁: 소셜 미디어 공유 미리보기가 예상대로 표시되는지 확인할 때는 다음 도구를 사용합니다. - Facebook/Open Graph: <https://developers.facebook.com/tools/debug/> - Twitter Card Validator: <https://cards-dev.twitter.com/validator> - kakaoTalk: <https://developers.kakao.com/tool/og>

베스트 프랙티스: `og:image` 는 절대 URL(<https://>로 시작)을 사용합니다. 상대 경로는 소셜 미디어 크롤러가 읽지 못합니다.

시맨틱 태그와 SEO의 연결

시맨틱 태그 사용이 SEO에 직접 도움이 되는 이유를 정리합니다.

```
<!-- 시맨틱 태그가 SEO에 미치는 영향 -->

<!-- h1: 가장 중요한 키워드를 포함합니다 -->
<h1>시맨틱 HTML로 만드는 접근성 있는 웹 페이지</h1>

<!-- article: 독립적인 콘텐츠를 구글이 인식합니다 -->
<article>
  <!-- h2: 주요 키워드를 포함한 섹션 제목 -->
  <h2>시맨틱 태그의 종류와 용도</h2>
  <p>
    <strong>header</strong>, <strong>nav</strong>, <strong>main</strong> 등의
    시맨틱 태그는 페이지 구조를 명확히 합니다.
  </p>
</article>

<!-- 구조화된 데이터 (JSON-LD): 검색 결과 리치 스니펫에 활용 -->
<script type="application/ld+json">
{
  "@context": "https://schema.org",
  "@type": "Article",
  "headline": "시맨틱 HTML 완전 정복",
  "author": {
    "@type": "Person",
    "name": "홍길동"
  },
  "datePublished": "2025-06-01"
}
</script>
```

줄별 코드 설명

줄	코드	설명
4	<code><h1>시맨틱 HTML로 만드는...</h1></code>	검색엔진은 <code><h1></code> 을 페이지 핵심 주제로 인식합니다. 검색하려는 키워드를 자연스럽게 포함시킵니다
8	<code><article></code>	구글은 <code><article></code> 태그를 독립적인 콘텐츠 단위로 인식하여 검색 결과에서 더 적합하게 처리합니다
10	<code><h2>시맨틱 태그의 종류와 용도</h2></code>	<code><h2></code> 에 포함된 키워드도 SEO에 영향을 줍니다. 자연스러운 문장으로 핵심 키워드를 포함합니다
16~26	<code><script type="application/ld+j..."></code>	JSON-LD 형식의 구조화 데이터입니다. 검색 결과에 별점, 날짜, 작성자 등이 표시되는 "리치 스니펫"을 만들 수 있습니다

절 요약

- SEO는 검색 결과 노출 최적화이며, HTML 구조와 메타 정보가 핵심 역할을 합니다
- `<title>` 은 50~60자, `meta description` 은 150~160자로 각 페이지마다 고유하게 작성합니다
- Open Graph 태그로 소셜 미디어 공유 미리보기를 제어합니다
- 시맨틱 태그, 제목 계층, alt 속성 모두 SEO와 접근성에 동시에 기여합니다

FAQ

Q1: `<div>` 에 클래스나 id로 이름을 붙이면 시맨틱 태그와 동일한 효과가 있지 않습니까?

A1: 그렇지 않습니다. `<div id="header">` 와 `<header>` 는 시각적으로는 차이가 없지만, 검색엔진과 스크린 리더는 HTML 태그 이름을 기준으로 의미를 해석합니다. `<div>` 는 어떤 의미도 갖지 않는 중립적인 컨테이너이므로, 클래스 이름이 무엇이든 기계는 그 의미를 파악하지 못합니다. 반면 `<header>` 태그는 자동으로 `banner` 랜드마크 역할을 가져 스크린 리더가 인식하고 검색엔진이 구조를 파악합니다.

Q2: ARIA를 많이 사용할수록 접근성이 더 좋아지는 것 아닌가요?

A2: 오히려 반대입니다. 이미 시맨틱 의미를 가진 요소에 중복된 ARIA를 붙이면 보조기기에 혼란을 줍니다. 예를 들어 `<nav role="navigation">` 처럼 이미 `navigation` 랜드마크 역할을 갖는 `<nav>` 에 또 `role="navigation"` 을 추가하는 것은 불필요합니다. ARIA는 시맨틱 HTML로 해결할 수 없는 경우에만 보완적으로 사용합니다. "No ARIA is better than bad ARIA"가 WAI-ARIA 명세의 기본 원칙입니다.

Q3: 스킵 내비게이션(skip navigation) 링크가 항상 필요합니까?

A3: 반복되는 내비게이션 메뉴가 있는 페이지라면 반드시 필요합니다. 키보드 사용자는 매 페이지마다 수십 개의 메뉴 링크를 탭으로 지나쳐야 본문에 도달합니다. 스킵 내비게이션 링크는 이 과정을 단 한 번의 탭으로 해결해 줍니다. 일반 사용자에게는 보이지 않도록 `.sr-only` CSS로 숨기고, 탭 키를 처음 누를 때만 나타나게 하는 것이 일반적인 구현 방식입니다.

Q4: `meta description` 이 검색 순위에 직접 영향을 줍니까?

A4: 검색 순위에는 직접적인 영향이 없지만, 클릭률(CTR)에 큰 영향을 미칩니다. 매력적인 description은 검색 결과에서 클릭을 유도하고, 높은 클릭률은 간접적으로 SEO 점수를 높입니다. 또한 검색엔진이 description을 무시하고 페이지 본문에서 직접 발췌하는 경우도 있으므로, description 작성과 함께 본문 텍스트의 질도 중요합니다.

Q5: `<section>` 과 `<div>` 를 어떻게 구분하여 사용합니까?

A5: 간단한 판단 기준은 "이 영역에 제목을 붙일 수 있는가?"입니다. 제목 태그(`<h2>`, `<h3>` 등)와 함께 주제를 나눌 수 있다면 `<section>` 이 적합합니다. 단순히 스타일링을 위한 묶음이거나 레이아웃 래퍼(wrapper) 역할이라면 `<div>` 를 사용합니다. `<section>` 안에는 항상 제목이 포함되어야 하고, 그렇지 않다면 `<div>` 가 더 적절합니다.

장 요약

이번 장에서는 HTML이 단순한 구조 마크업을 넘어 의미(Semantics)를 전달하는 언어임을 배웠습니다. 시맨틱 태그(`<header>`, `<nav>`, `<main>`, `<article>`, `<aside>`, `<footer>`)를 사용하면 검색엔진이 페이지 구조를 정확히 파악하고, 스크린 리더 사용자가 랜드마크와 제목 계층을 통해 페이지를 효율적으로 탐색할 수 있습니다. 웹 접근성은 소수를 위한 배려가 아니라 모든 사용자의 경험을 향상시키는 설계 원칙이며, ARIA는 시맨틱 HTML이 표현하지 못하는 동적 UI 상태를 보조기기에 전달하는 보완 도구입니다. 마지막으로 `<title>`, `meta`

`description`, Open Graph 태그를 통해 검색 결과와 소셜 공유 미리보기를 제어하는 SEO 기
초까지 익혔습니다.

핵심 키워드

`시맨틱 태그`, `랜드마크`, `문서 아웃라인`, `웹 접근성(a11y)`, `스크린 리더`, `키보드 내비게이
션`, `ARIA`, `aria-label`, `aria-hidden`, `SEO`, `Open Graph`

다음 장 미리보기

5장에서는 HTML로 만든 구조에 CSS를 입혀 시각적으로 디자인하는 방법을 배웁니다. CSS 선
택자로 원하는 요소를 정확히 지정하고, 박스 모델로 요소의 크기와 여백을 제어하는 원리를
이해하게 됩니다.

✂ 실습 프로젝트 (Hands-on Project)

4장 실습 프로젝트: 시맨틱 블로그 글 페이지

이 장에서 배운 개념을 실무 시나리오에 적용하는 프로젝트입니다.

초급: 시맨틱 구조 블로그 글 페이지

시나리오

여러분은 기술 블로그를 운영하려 합니다. 블로그 글 한 편을 HTML로 작성하는데,
단순히 `div`만 쓰는 대신 시맨틱 태그로 구조를 잡고 기본 접근성까지 갖춘 페이지를
만들어야 합니다. SEO를 위한 기본 메타 태그도 포함합니다.

학습 목표

- `header`, `nav`, `main`, `article`, `aside`, `footer` 로 페이지 구획을 설계할 수 있습
니다
- 올바른 제목 계층(`h1` → `h2` → `h3`)을 구성할 수 있습니다
- `alt`, `label`, 포커스 스타일 등 기본 접근성을 적용할 수 있습니다

진행 방법

1. `project_starter.html` 을 열고 TODO 주석을 찾습니다
2. 각 TODO를 4장에서 배운 시맨틱 태그와 접근성 개념으로 완성합니다
3. 완성 후 브라우저에서 열어 결과를 확인합니다
4. Tab 키로 탐색하며 포커스 이동이 올바른지 확인합니다

실행 방법

브라우저에서 `project_starter.html` 파일을 직접 열거나 VS Code Live Server로 실행합니다.

완성 기준

- [] `<div>` 없이 시맨틱 태그만으로 페이지 구획이 구성되어 있다
 - [] `h1` 이 하나이며, `h1` → `h2` → `h3` 계층이 건너뛰어 없이 연결된다
 - [] `<img>` 에 `alt` 속성이 있다 (의미 있는 이미지는 설명, 장식은 `alt=""`)
 - [] `<label>` 이 입력 필드와 연결되어 있다
 - [] 포커스 스타일(`outline`)이 제거되지 않았다
 - [] `<title>` 과 `<meta name="description">` 이 올바르게 작성되어 있다
-

중급: 접근성과 SEO를 갖춘 블로그 글 페이지

시나리오

초급에서 만든 페이지를 실무 수준으로 업그레이드합니다. 스킵 내비게이션, ARIA 속성, Open Graph 메타 태그, JSON-LD 구조화 데이터까지 실제 서비스에 배포 가능한 수준의 완성도를 목표로 합니다.

학습 목표

- 스킵 내비게이션(`skip navigation`)으로 키보드 접근성을 향상시킬 수 있습니다
- `aria-label` , `aria-hidden` , `aria-expanded` 등 ARIA 속성을 적절히 사용할 수 있습니다
- Open Graph와 Twitter Card 메타 태그로 소셜 공유 미리보기를 설정할 수 있습니다
- JSON-LD 구조화 데이터로 검색 결과 리치 스니펫을 준비할 수 있습니다

진행 방법

1. `project_advanced_starter.html` 을 열고 TODO 주석을 찾습니다
2. 초급에서 연습한 내용을 바탕으로 더 복잡한 접근성 및 SEO 요구사항을 완성합니다
3. 완성 후 브라우저에서 열어 결과를 확인합니다
4. Tab 키로 탐색하며 스킵 내비게이션, 드롭다운 버튼 상태 등을 확인합니다

실행 방법

브라우저에서 `project_advanced_starter.html` 파일을 직접 열거나 VS Code Live Server로 실행합니다.

완성 기준

- [] 스킵 내비게이션 링크가 있으며, Tab 키를 처음 누를 때 나타난다
- [] `<nav>` 에 `aria-label` 이 있다
- [] 아이콘 버튼에 `aria-label` 이 있다
- [] 드롭다운 버튼에 `aria-expanded` 가 있고, JavaScript로 상태가 토글된다
- [] `aria-live` 영역이 있어 동적 메시지를 스크린 리더에 알릴 수 있다
- [] Open Graph(`og:*`) 메타 태그가 절대 URL로 완성되어 있다
- [] JSON-LD 구조화 데이터가 포함되어 있다

확장 아이디어

- 목차 링크 클릭 시 해당 섹션으로 부드럽게 스크롤하는 기능 추가하기 (`scroll-behavior: smooth` CSS 속성 활용)
- `prefers-reduced-motion` 미디어 쿼리로 모션 감소 설정을 존중하는 코드 추가하기
- 다국어 지원을 고려하여 `hreflang` 메타 태그 구조 설계하기

▶  `project_starter.html` (스타터 코드 - 직접 완성해보세요)

▶  `project_complete.html` (완성 코드)

▶  project_advanced_starter.html (스타터 코드 - 직접 완성해보세요)

▶  project_advanced_complete.html (완성 코드)